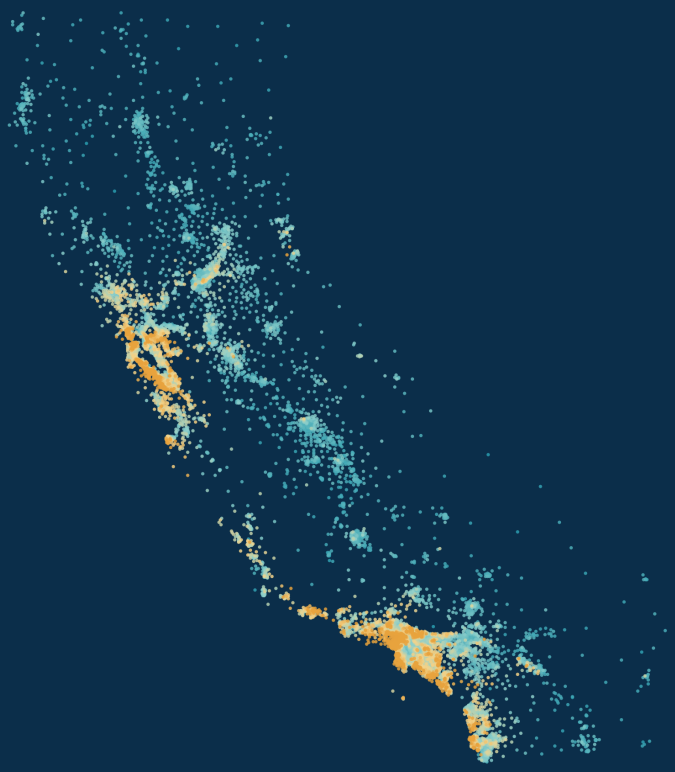
TECHNICAL REPORT

California House Price Prediction Using Machine Learning

End-to-end supervised regression workflow with a Streamlit prediction application



Census block groups (housing.csv) by median value

AUTHOR

Alwin V J

PROJECT CATEGORY

Machine Learning — Regression

MAIN TECHNOLOGIES

Python, Pandas, NumPy, Scikit-learn, Joblib, Streamlit

REPORT DATE

15 October 2026

Based on the actual project implementation (notebook, dataset, saved model and Streamlit app).

Table of Contents

1 Cover Page . . . 1
2 Executive Summary . . . 3
3 Introduction and Problem Statement . . . 3
4 Project Objectives and Scope . . . 4
5 Dataset Description . . . 4
6 Data Preprocessing . . . 6
6.1 Missing-value handling . . . 6
6.2 Numerical feature scaling . . . 6
6.3 Categorical encoding . . . 6
6.4 Pipeline architecture . . . 6
6.5 Train–test split . . . 6
7 Exploratory Data Analysis . . . 7
Interpreting the correlations . . . 8
8 Baseline Model . . . 8
9 Model Comparison and Cross-Validation . . . 9
10 Hyperparameter Tuning . . . 9
Implementation note 1 — final model configuration . . . 10
11 Final Model Evaluation . . . 10
Interpretation . . . 11
Residual analysis . . . 11
12 Model Serialization and Inference . . . 12
Inference workflow . . . 13
13 Streamlit Application . . . 13
Implementation note 2 — prediction output units . . . 14
Implementation note 3 — input handling . . . 14
14 Limitations and Future Improvements . . . 14
Limitations . . . 14
Proposed future work (not implemented) . . . 15
15 Conclusion . . . 15
16 References . . . 15
A Appendix: Verification Log and Technical Inconsistencies . . . 16

2 Executive Summary

This report documents **California House Price Prediction Using Machine Learning**, a supervised regression project that predicts the **median house value of a California census block group** (target `median_house_value`, in US dollars) from location, housing, population, income and ocean-proximity features. Every figure below is taken from the project's notebook outputs or from direct inspection of the project files; where the files do not establish a fact, the report says so.

Dataset. The California Housing Prices dataset (`housing.csv`) holds 20,640 observations and 10 columns: eight numerical inputs, one categorical input (`ocean_proximity`) and the numerical target. Each row is a census block group, not an individual house. Only `total_bedrooms` has missing values (207).

Workflow. The notebook performs exploratory analysis, builds a scikit-learn preprocessing pipeline (median imputation and standardization for numerical features; most-frequent imputation and one-hot encoding for the categorical feature), holds out 20% of the data as a test set, establishes a Linear Regression baseline, compares five regressors with shuffled five-fold cross-validation, tunes the best model with `GridSearchCV`, evaluates the final pipeline on the held-out test set and saves it with Joblib. A Streamlit application loads the saved pipeline for interactive prediction.

Model selection. HistGradientBoostingRegressor had the lowest cross-validation RMSE (48,140.81 USD) among the five models and was selected and tuned.

Table 1. Headline results (USD except R^2). Test-set values are from the held-out 20% split.

Model / evaluation	RMSE	MAE	R^2
Final HistGradientBoosting — **test set**	47,319.30	31,253.32	0.829
Final HistGradientBoosting — training set	36,250.50	24,604.66	0.902
Linear Regression baseline — test set	70,059.19	50,670.49	0.625

Application. `app.py` provides a Streamlit form for the nine inputs and displays the predicted median house value. The project documents local execution only; public deployment could not be verified.

Two implementation issues were found and are documented, not corrected

(1) The final model is built with `l2_regularization = 0.1`, whereas GridSearchCV reported 0.0 as best (Section 10). (2) The Streamlit app multiplies the model output by 100,000 although the model already predicts US dollars, so displayed values are inflated by a factor of 100,000 (Section 13). The original project files were not modified.

3 Introduction and Problem Statement

The objective is to predict the median house value of a California census block group from the characteristics available for that block group: geographic coordinates, the median age of its houses, the total number of rooms and bedrooms, its population and number of households, its median household income and its proximity to the ocean.

Why regression. The target is a continuous number measured in US dollars, so the task is supervised regression. Performance is judged by how far predictions fall from the true values on data the model has not seen (RMSE and MAE, in dollars) and by how much target variance is explained (R^2).

What is being predicted. The target is the *median house value of a census block group*. It is not the price of an individually specified property: variables such as `total_rooms`, `population` and `households` are block-group totals. Predictions therefore describe an area, not a particular house.

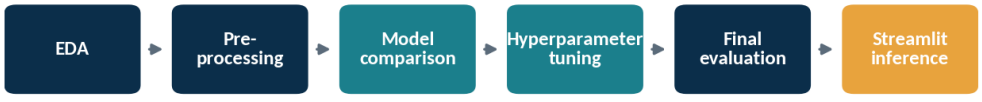


Figure 1. Project workflow implemented in the notebook and application.

4 Project Objectives and Scope

The project aims to practise a complete, production-oriented regression workflow rather than to train a single model. Table 2 separates work that exists in the project from the proposed improvements covered in Section 14.

Table 2. Completed work and where it is evidenced in the project files.

Activity	Status	Evidence
Exploratory data analysis	Completed	Notebook: shape, dtypes, missing values, duplicates, statistics, distributions, boxplots, correlations
Data preprocessing	Completed	Notebook: ColumnTransformer with numerical and categorical pipelines
Train–test splitting	Completed	Notebook: 80% / 20%, random_state = 42
Baseline model	Completed	Notebook: Linear Regression, train and test metrics
Cross-validation and model comparison	Completed	Notebook: five models, shuffled 5-fold CV
Hyperparameter tuning	Completed	Notebook: GridSearchCV, 243 candidates
Final evaluation and residual analysis	Completed	Notebook: train/test metrics, two residual plots
Model serialization	Completed	Notebook: joblib.dump; file house_price_model.joblib
Streamlit-based inference	Completed (local)	app.py; no public deployment verified
Feature engineering, uncertainty, tests, versioning, deployment	**Proposed**	Not implemented (README “Future Improvements”)

5 Dataset Description

Name and source. California Housing Prices, file `housing.csv`. The notebook and README cite the Kaggle page `kaggle.com/datasets/camnugent/california-housing-prices`. The original collection method and time period are not stated in the project files and could not be verified; the README describes the data only as historical census block-group information.

Structure. 20,640 rows and 10 columns: eight numerical input features, one categorical input feature (`ocean_proximity`) and one numerical target (`median_house_value`). All numerical columns are `float64`; `ocean_proximity` is a string column. The notebook found 0 duplicate rows and missing values only in `total_bedrooms` (207 of 20,640 rows, 1.0%).

Table 3. Feature descriptions. Ranges come from the notebook’s descriptive statistics and housing.csv.

Feature	Type	Meaning	Observed range / levels
longitude	float64	Geographic position, west–east. Values are negative: more negative = further west	−124.35 to −114.31
latitude	float64	Geographic position, south–north	32.54 to 41.95
housing_median_age	float64	Median age of houses in the block group (years)	1 to 52
total_rooms	float64	Total number of rooms in the block group	2 to 39,320
total_bedrooms	float64	Total number of bedrooms in the block group (207 missing)	1 to 6,445
population	float64	Total population of the block group	3 to 35,682
households	float64	Number of households (people sharing a home unit)	1 to 6,082
median_income	float64	Median household income, in **tens of thousands of US dollars**	0.50 to 15.00
ocean_proximity	string	Location relative to the ocean (categorical)	5 levels (Table 5)
median_house_value	float64	**Target**: median house value, US dollars	14,999 to 500,001

Table 4. Descriptive statistics of the numerical columns (notebook output; target in USD).

Column	Count	Mean	Std	Min	25%	50%	75%	Max
longitude	20,640	−119.570	2.004	−124.350	−121.800	−118.490	−118.010	−114.310
latitude	20,640	35.632	2.136	32.540	33.930	34.260	37.710	41.950
housing_median_age	20,640	28.639	12.586	1	18	29	37	52
total_rooms	20,640	2,635.763	2,181.615	2	1,447.75	2,127	3,148	39,320
total_bedrooms	20,433	537.871	421.385	1	296	435	647	6,445
population	20,640	1,425.477	1,132.462	3	787	1,166	1,725	35,682
households	20,640	499.540	382.330	1	280	409	605	6,082
median_income	20,640	3.871	1.900	0.500	2.563	3.535	4.743	15.000
median_house_value	20,640	206,855.82	115,395.62	14,999	119,600	179,700	264,725	500,001

Table 5. *ocean_proximity levels (counts computed from housing.csv).*

Level	Block groups	Share	Median target (USD)
<1H OCEAN	9,136	44.3%	214,850
INLAND	6,551	31.7%	108,500
NEAR OCEAN	2,658	12.9%	229,450
NEAR BAY	2,290	11.1%	233,800
ISLAND	5	0.02%	414,700

The ISLAND level has only five rows, so any pattern the model learns for it rests on very little data.

Target distribution. The target has mean 206,855.82 and median 179,700 USD, ranges from 14,999 to 500,001, and is right-skewed (skewness 0.98, computed from housing.csv). The notebook's value counts show 965 block groups (4.68%) at exactly 500,001, an upper cap (Figure 4).

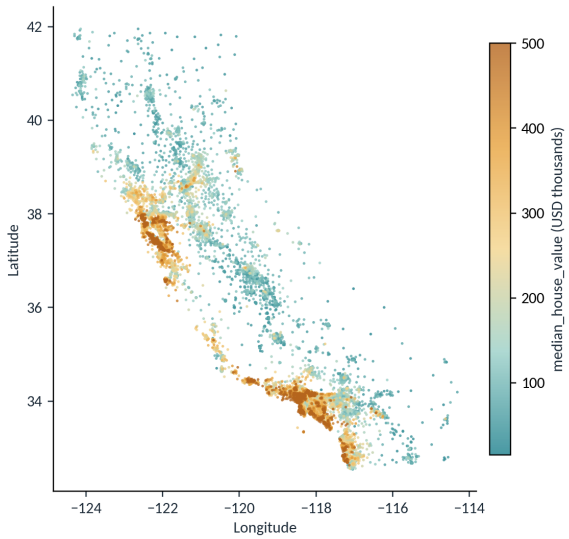


Figure 2. *Geographic distribution of the 20,640 block groups coloured by median house value (generated from housing.csv for this report; not a notebook output).*

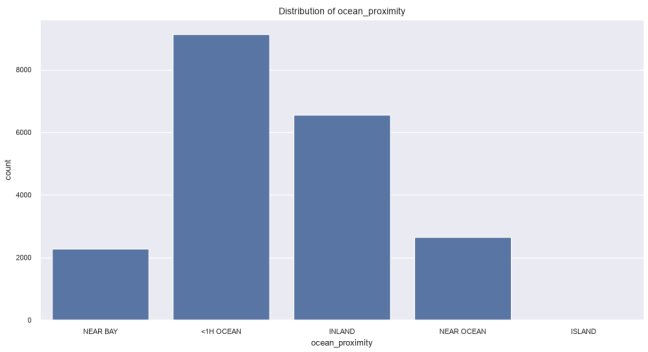


Figure 3. *Count of block groups per ocean_proximity level (notebook output).*

6 Data Preprocessing

The notebook separates features from the target, splits the data, and defines one preprocessing object that is reused by every model. The relevant notebook code is reproduced below.

```
numerical_transformer = Pipeline(steps=[("imputer", SimpleImputer(strategy="median")),
                                        ("scaler",  StandardScaler())])
categorical_transformer = Pipeline(steps=[("imputer", SimpleImputer(strategy="most_frequent")),
                                          ("onehot",  OneHotEncoder(handle_unknown="ignore"))])
preprocess = ColumnTransformer(transformers=[("num", numerical_transformer, numerical_features),
                                             ("cat", categorical_transformer, categorical_features)])
```

6.1 Missing-value handling

`total_bedrooms` has 207 missing values; no other column has any. Numerical features are imputed with the **median**, which is robust to the heavy right tails seen in Section 7. The categorical feature is imputed with its **most frequent** category; `ocean_proximity` has no missing values in this dataset, so that step acts as a safeguard for new data.

6.2 Numerical feature scaling

`StandardScaler` rescales each numerical feature to zero mean and unit variance. Features here differ by orders of magnitude (for example `total_rooms` up to 39,320 versus `median_income` up to 15), which matters for linear models and for the regularization penalties in Ridge and Lasso. Scaling is not required by every model: tree-based models such as Random Forest and HistGradientBoosting are insensitive to monotonic rescaling of features. The notebook nevertheless applies the same preprocessor to all five models for simplicity.

6.3 Categorical encoding

`OneHotEncoder` turns the five `ocean_proximity` levels into five binary columns, giving 13 model inputs in total (8 numerical + 5 binary; confirmed in the saved model). With `handle_unknown="ignore"`, a category not seen during training is encoded as all zeros instead of raising an error. This keeps inference from failing, but it also means an invalid category is silently accepted (see Section 13).

6.4 Pipeline architecture

`ColumnTransformer` routes numerical and categorical columns to their own transformers; a scikit-learn `Pipeline` chains it with the regressor. Because cross-validation, grid search and the saved artifact all use the whole pipeline, the imputer, scaler and encoder are re-fitted on each training fold and are applied identically at prediction time. This supports consistent training, validation and inference and prevents statistics from validation or test rows leaking into preprocessing.

6.5 Train–test split

`train_test_split(X, y, test_size=0.2, random_state=42)` produces 16,512 training rows and 4,128 test rows, without stratification. The test set must stay out of model selection and tuning so that its error is an honest estimate for unseen data. The notebook follows this: cross-validation and `GridSearchCV` are fitted on `X_train, y_train` only, and the test set is used for evaluation of the fitted models.

7 Exploratory Data Analysis

The EDA covers dataset structure (shape, dtypes), missing values, duplicates, descriptive statistics (Table 4), feature distributions, outliers, the target distribution and correlations. Main findings:

- **Missing values:** 207 in `total_bedrooms` (Section 5).
- **Capped target:** the maximum is 500,001, shared by 965 block groups; the histogram shows a spike at the upper end (Figure 4). Values near the cap are therefore censored rather than genuine market maxima.
- **Skew and extreme values:** total_rooms, total_bedrooms, population and households are strongly right-skewed with extreme maxima (Figures 5 and 6).
- **Correlations with the target** (Table 6, Figure 7): median_income is the strongest at +0.688; latitude is −0.144; total_rooms is +0.134.
- **Collinearity:** total_rooms, total_bedrooms, population and households are strongly inter-correlated (pairwise 0.86 to 0.98 in housing.csv), and longitude and latitude correlate at −0.92 (heatmap).

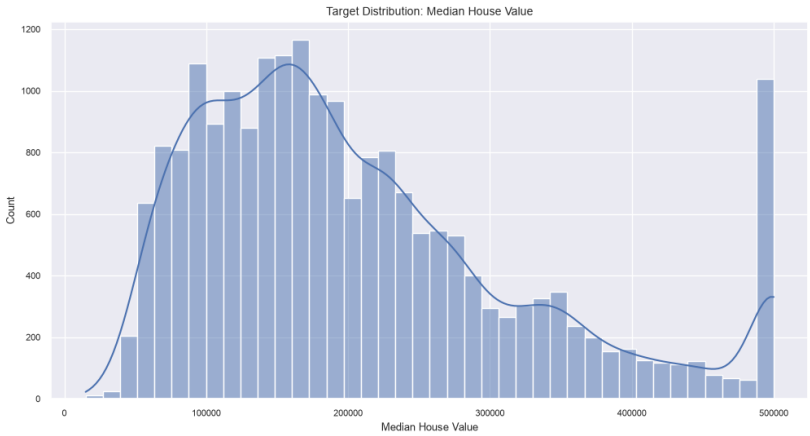


Figure 4. *Distribution of median_house_value with the upper cap near 500,001 (notebook output).*

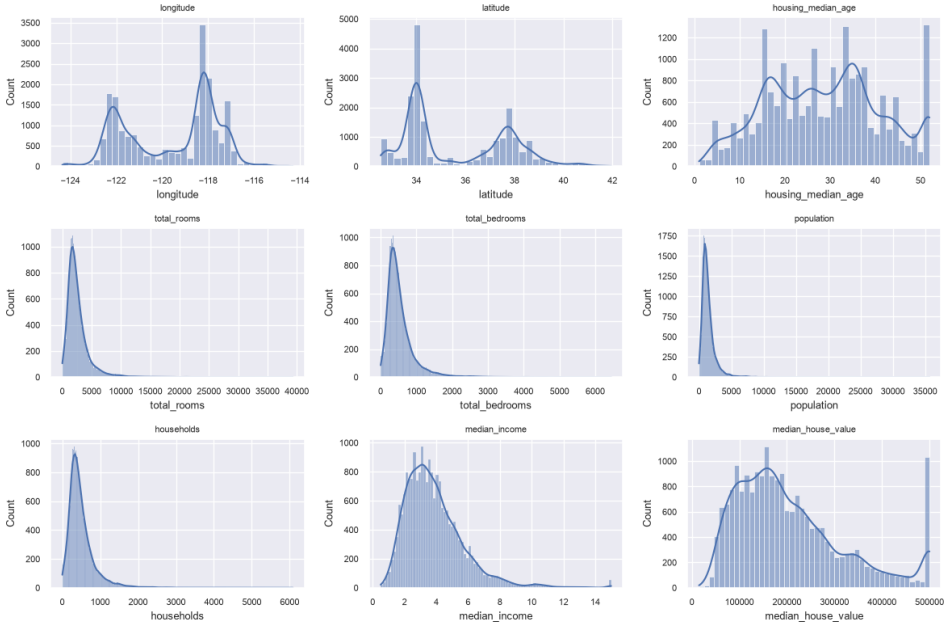


Figure 5. *Histograms of the nine numerical columns (notebook output).*

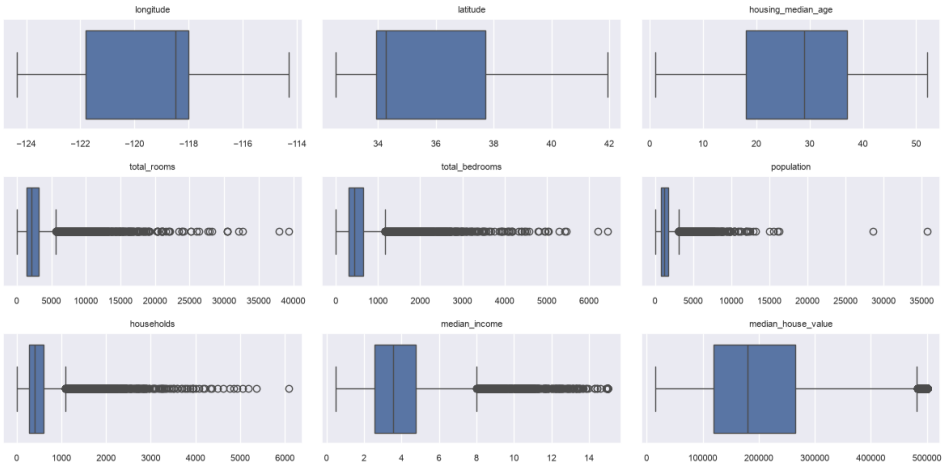


Figure 6. *Boxplots of the nine numerical columns showing outliers (notebook output).*

Table 6. *Pearson correlation of each numerical column with median_house_value (notebook output).*

Feature	Correlation	Feature	Correlation
median_income	0.688	population	−0.025
total_rooms	0.134	longitude	−0.046
housing_median_age	0.106	latitude	−0.144
households	0.066	total_bedrooms	0.050

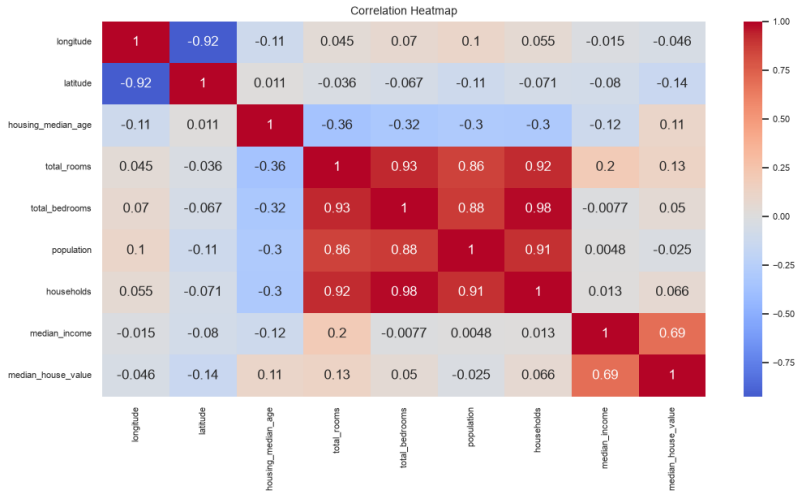


Figure 7. *Correlation heatmap of the numerical columns (notebook output).*

Interpreting the correlations

Pearson correlation measures *linear* association between two variables and does not establish causation. In geographically structured housing data it is especially limited: location effects are non-linear and interact (coastal versus inland areas, metropolitan clusters, Figure 2), so a weak linear correlation for latitude or longitude does not mean location is unimportant. Strongly correlated count features also share information, which makes individual coefficients or pairwise correlations unreliable as measures of importance. The notebook's qualitative notes (capped target, skew, multicollinearity, income as the strongest predictor) are consistent with the figures above.

8 Baseline Model

A `LinearRegression` pipeline (same preprocessor) was fitted on the training split without cross-validation or tuning. A baseline gives a reference point: a more complex algorithm is only justified if it clearly beats a simple, interpretable model on the same data and metrics.

Table 7. *Linear Regression baseline (single train/test split).*

Set	RMSE (USD)	MAE (USD)	R^2
Training	68,433.94	49,594.84	0.650
Test	70,059.19	50,670.49	0.625

Training and test scores are close, which suggests the linear model is limited by its capacity (underfitting) rather than by variance. The baseline test metrics were computed before the cross-validated comparison; model selection and tuning relied only on cross-validation within the training split.

9 Model Comparison and Cross-Validation

Five regressors were compared with `KFold(n_splits=5, shuffle=True, random_state=42)` applied to the training split (16,512 rows), each wrapped in the same preprocessing pipeline. The scoring metrics were negative RMSE, negative MAE and R^2, averaged over the five folds. RMSE is the primary selection metric; MAE and R^2 are secondary. The models were: LinearRegression; Ridge and Lasso (random_state = 42; Lasso with max_iter = 10000, other settings default); RandomForestRegressor and HistGradientBoostingRegressor, both with default hyperparameters and no random_state set.

Table 8. *Five-fold cross-validation results on the training split (not test-set results).*

Model	CV RMSE (USD)	CV MAE (USD)	CV R^2
Linear Regression	68,604.16	49,667.16	0.648
Ridge Regression	68,595.62	49,664.33	0.648
Lasso Regression	68,603.23	49,667.26	0.648
Random Forest	49,344.98	32,214.17	0.818
HistGradientBoosting	**48,140.81**	**32,193.56**	**0.827**

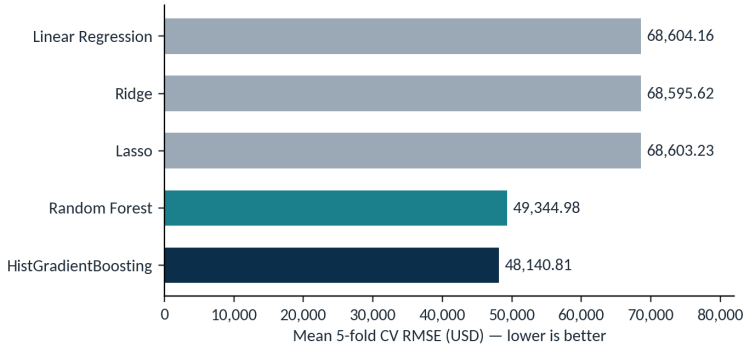


Figure 8. *Mean five-fold cross-validation RMSE by model (lower is better; values from Table 8).*

Selection. HistGradientBoosting has the lowest CV RMSE (48,140.81), and also the lowest CV MAE and highest CV R^2, so it was selected. Linear, Ridge and Lasso are practically identical (about 68.6k RMSE): regularization does not help here. The two tree ensembles lower RMSE by roughly 30% relative to the linear models, which indicates non-linear structure. Because Random Forest and HistGradientBoosting were run without a fixed random_state, exact values may differ slightly on re-execution.

10 Hyperparameter Tuning

`GridSearchCV` tuned the HistGradientBoostingRegressor pipeline (random_state = 42) with the same shuffled five-fold split and the scoring `neg_root_mean_squared_error`. The grid has 3 values for each of 5 parameters, i.e. 243 candidates and 1,215 fits.

Table 9. *Parameter grid (notebook).*

Parameter	Candidate values
learning_rate	0.03, 0.05, 0.1
max_depth	None, 3, 6

Parameter	Candidate values
max_leaf_nodes	15, 31, 63
min_samples_leaf	20, 50, 100
l2_regularization	0.0, 0.1, 1.0

Best GridSearchCV result (notebook output): CV RMSE **47,327.97** with learning_rate = 0.1, max_depth = None, max_leaf_nodes = 63, min_samples_leaf = 20 and l2_regularization = 0.0. Four of the best values lie on the edge of the searched ranges (learning_rate, max_depth, max_leaf_nodes, min_samples_leaf), so a wider grid could find better settings.

Implementation note 1 — final model configuration

After the search, the notebook (section "Retraining with best params") does not use `grid.best_estimator_`; it builds the final pipeline by hand:

```
hgb_best = Pipeline(steps=[("preprocess", preprocess),
    ("model", HistGradientBoostingRegressor(l2_regularization=0.1, learning_rate=0.1,
                                            max_depth=None, max_leaf_nodes=63, min_samples_leaf=20))])
hgb_best.fit(X_train, y_train)
```

Table 10. GridSearchCV best parameters versus the model actually fitted and saved.

Parameter	GridSearchCV best	Final model (notebook code and saved Joblib)
learning_rate	0.1	0.1
max_depth	None	None
max_leaf_nodes	63	63
min_samples_leaf	20	20
l2_regularization	**0.0**	**0.1**
random_state	42	not set (None)

What the code actually does

The final estimator differs from the GridSearchCV winner in **l2_regularization** (0.1 instead of 0.0) and does not set **random_state**. The value 0.1 was one of the grid candidates, but it was not the best-scoring one. The saved Joblib file confirms l2_regularization = 0.1. The CV RMSE of 47,327.97 therefore belongs to the l2 = 0.0 candidate and is not a measured score of the final estimator. The README states that the "final selected configuration" uses l2_regularization = 0.1; the notebook output shows the search returned 0.0, and per the project's own rule the notebook takes precedence.

How much does it matter? The notebook did not store the cross-validation score of the l2 = 0.1 candidate. As supplementary evidence only, the two configurations were re-run in a separate environment (scikit-learn 1.8.0, same data split, same KFold, random_state = 42): CV RMSE 47,465.63 for l2 = 0.0 and 47,408.36 for l2 = 0.1. These values do not reproduce the notebook's 47,327.97 (the saved model was serialized with scikit-learn 1.9.0) and are not used elsewhere in this report, but they suggest that the two settings are practically tied (about 0.1% apart). The final test metrics in Section 11 are those of the l2 = 0.1 model actually trained and saved.

Early stopping and reproducibility. With more than 10,000 training rows, `HistGradientBoostingRegressor` enables early stopping automatically (`early_stopping="auto"`, 10% internal validation split). The saved model has early stopping active and 100 fitted iterations; because no random_state was set, retraining can produce slightly different results from the saved artifact.

11 Final Model Evaluation

Table 11. Final model (HistGradientBoosting pipeline) — notebook results.

Metric	Training set	Test set	Baseline LR (test)
RMSE (USD)	36,250.50	47,319.30	70,059.19

Metric	Training set	Test set	Baseline LR (test)
MAE (USD)	24,604.66	31,253.32	50,670.49
R^2	0.902	0.829	0.625

Interpretation

- **RMSE** is the square root of the mean squared prediction error, in US dollars. Squaring makes large misses count more. Test RMSE ≈ **$47,319**, about 22.9% of the mean target (206,856).
- **MAE** is the average absolute error in dollars: on the test set predictions are off by about **$31,253** on average (about 15.1% of the mean target).
- **R^2** is the share of target variance explained relative to always predicting the mean. The test R^2 of 0.829 means about 83% of the variance is explained; 17% is not.

Versus the baseline. On the test set the final model lowers RMSE from 70,059.19 to 47,319.30 (−32.5%) and MAE from 50,670.49 to 31,253.32 (−38.3%), and raises R^2 from 0.625 to 0.829.

Training versus test. Test RMSE is about $11,069 (30.5%) above training RMSE and R^2 falls from 0.902 to 0.829. Some gap is expected for any flexible model scored on rows it was fitted to. The cross-validated RMSE of the tuned search (47,327.97, from the training split only) is very close to the test RMSE, which indicates that test performance is in line with what validation predicted. The gap by itself therefore does not demonstrate harmful overfitting, but the evidence also cannot rule out some loss of generalization, and no external dataset was used.

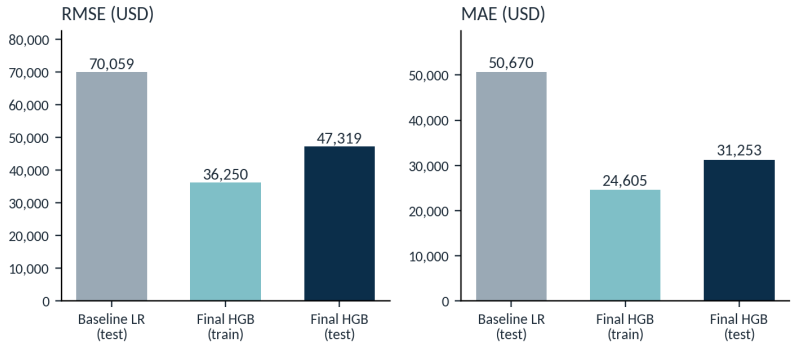


Figure 9. Baseline test error versus final-model training and test error (values from Table 11).

Residual analysis

The notebook plots residuals (actual minus predicted) for the test set. Visible features, read from the plots:

- The spread of residuals **widens as predicted value increases**, so errors are larger for expensive block groups (Figure 10).
- A **straight diagonal band** of points runs down to the right at high predictions. These are block groups whose true value is the 500,001 cap: their residual equals 500,001 minus the prediction, and some predictions exceed 500,000. The model has no built-in awareness of the cap.
- The residual histogram is **centred near zero with a heavier right tail** (Figure 11), i.e. some block groups are substantially under-predicted. Residuals reach roughly −270,000 to +315,000 USD.

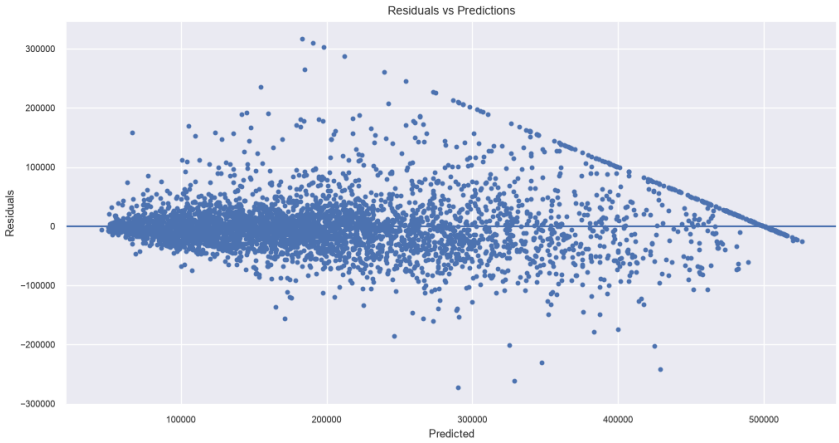


Figure 10. *Residuals versus predictions on the test set (notebook output).*

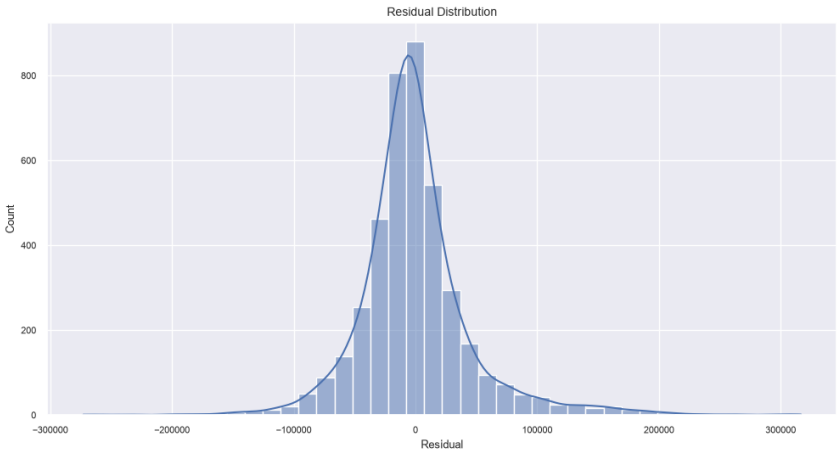


Figure 11. *Distribution of test-set residuals (notebook output).*

These plots do not support a claim that the model is unbiased across the value range, and nothing in the project supports calling it production-ready.

12 Model Serialization and Inference

The notebook's last cell saves the fitted pipeline with `joblib.dump(hgb_best, "house_price_model.joblib")`. The artifact is the **whole pipeline** (preprocessor plus model), so imputation medians, scaling statistics, encoder categories and trees are stored together and inference applies exactly the transformations used in training. The Streamlit app loads it with `joblib.load` and needs no retraining.

Inspection method. Joblib files are Python pickles and can execute code when loaded, so the file was first inspected with an unpickler restricted to scikit-learn, NumPy and Joblib classes only; no other classes were present. Original project files were never modified (checksums were verified before and after).

Table 12. Contents of house_price_model.joblib.

Property	Value found in the artifact
File size	743,162 bytes
Object	sklearn Pipeline: preprocess (ColumnTransformer) → model (HistGradientBoostingRegressor)
Serialized with	scikit-learn 1.9.0
Expected input	9 columns: the eight numerical features and ocean_proximity; 13 features after preprocessing
Numerical steps	SimpleImputer (median), StandardScaler
Categorical steps	SimpleImputer (most_frequent, learned <1H OCEAN), OneHotEncoder(handle_unknown="ignore"); levels <1H OCEAN, INLAND, ISLAND, NEAR BAY, NEAR OCEAN
Model parameters	loss squared_error, learning_rate 0.1, max_iter 100, max_leaf_nodes 63, max_depth None, min_samples_leaf 20, l2_regularization 0.1, early_stopping auto, random_state None
Fitted state	100 trees; early stopping active; median imputer value for total_bedrooms = 437.0 (full-dataset median 435), consistent with fitting on the 80% training split

Inference workflow

1. Collect the nine feature values (in the notebook, via `predict_house_price(...)`; in the app, via form widgets).
2. Build a one-row Pandas DataFrame whose column names match the training data.
3. Call `model.predict(new_row)[0]`: the pipeline imputes, scales and one-hot encodes, then the boosted trees predict.
4. The result is the predicted median house value **in US dollars**.

Verification. For the first dataset row (actual value 452,600) the notebook prints 466,109.45; loading the saved file independently reproduced 466,109.4476 (scikit-learn 1.9.0, and also scikit-learn 1.7.2, which emits an `InconsistentVersionWarning`).

13 Streamlit Application

`app.py` implements the front end. **Page configuration:** title “California House Price Predictor”, house icon, wide layout, plus custom CSS for a dark card-style interface. **Model loading:** `load_model()` returns `joblib.load("house_price_model.joblib")` (a path relative to the working directory) and is decorated with `@st.cache_resource`, so the model is loaded once per server process. **Prediction:** the “Predict House Price” button builds a one-row DataFrame, calls `model.predict()` and shows the result with `st.metric` as “Predicted Median House Value”; exceptions are reported with `st.error`.

Table 13. Input fields in app.py, grouped as on the page.

Group	Field	Widget and constraints	Default
Location	Longitude	number input, format %.2f	−124.35
Location	Latitude	number input, format %.2f	32.54
Housing Information	Housing Median Age (years)	integer, min 1, step 1	1
Housing Information	Total Rooms	integer, min 1	1
Housing Information	Total Bedrooms	integer, min 1	1
Population Information	Population	integer, min 0	0
Population Information	Households	integer, min 0	0
Population Information	Median Income (in $10,000s)	float, min 0.0, step 0.1, format %.2f	0.00
Ocean Proximity	Ocean Proximity	select box: placeholder text plus 5 levels	placeholder



Figure 12. Architecture of the application workflow. This is an illustrative diagram, not a screenshot: no browser was available in the inspection environment, so the running interface could not be captured.

Running locally (from the README and requirements.txt): create the environment with `conda create -n cali_house python=3.12` and activate it; run `pip install -r requirements.txt` (pinned: streamlit 1.50.0, pandas 2.3.3, numpy 2.3.3, scikit-learn 1.7.2, joblib 1.5.2); then, from the project folder, run `python -m streamlit run app.py`. The project contains no deployment configuration and no public URL; **public deployment is not verified**, and the README gives only local instructions.

The app was executed headlessly (Streamlit's AppTest on a copy of the project, scikit-learn 1.9.0) to observe real behaviour. It runs without exceptions.

Implementation note 2 — prediction output units

In `app.py` the prediction is converted with `predicted_price = prediction * 100000`, under the comment "Target is expressed in units of $100,000". That does not match this project:

Table 14. Evidence on the scale of the model output.

Source	Observation
housing.csv / notebook	median_house_value is in US dollars (e.g. 452,600; maximum 500,001). The model is trained on these raw values (no target scaling in the notebook).
Notebook inference example	predict_house_price(...) for the first row returns 466,109.45, i.e. about $466 thousand, consistent with the actual 452,600.
Saved model	Loading the artifact and predicting the same row gives 466,109.4476.
Streamlit app (headless run)	Same inputs, "NEAR BAY": displayed value **$46,610,944,761.88** (466,109.4476 × 100,000).

Conclusion: the ×100,000 conversion is inconsistent with the saved model

The model already outputs dollars, so the app overstates every prediction by a factor of 100,000 (about $46.6 billion for the example row). A factor of 100,000 would be appropriate for a target stored in units of $100,000, as in some other California-housing sources; this is a plausible origin of the error but could not be verified from the project. The original code was **not** altered. The expected fix is to display the model output directly.

Implementation note 3 — input handling

The select box's first option is the text "Select the proximity level from the drop down list", but the code only tests `ocean_proximity == "None"`, which never matches. The placeholder string is therefore passed to the model, where `handle_unknown="ignore"` encodes it as all zeros, and a prediction is returned silently. The default numeric values (population 0, households 0, median income 0.0, latitude and longitude at their dataset minima) lie outside the observed data; a prediction with the untouched defaults displays $30,793,383,904.90. The form also has no way to leave total_bedrooms missing although the pipeline could impute it.

14 Limitations and Future Improvements

Limitations

- **Historical data:** the dataset is census block-group data from an unspecified period; housing markets change over time.
- **Target ceiling:** 965 block groups (4.68%) are capped at 500,001, so the model cannot learn true values above the cap and tends to mis-estimate the most expensive areas.

- **Distribution shift:** performance on other regions, time periods or data collected differently is unknown; the ISLAND level has only five rows.
- **Limited features:** only nine block-group aggregates are available; no property-level characteristics (size, condition, lot), amenities or time information.
- **Aggregate versus individual:** predictions describe a block group's median value and should not be read as the valuation of a single property.
- **No external validation:** all results come from one random 80/20 split of one dataset; no external dataset or temporal/spatial hold-out was tested.
- **Open issues** described in Sections 10 and 13 (final l2 setting, ×100,000 output scaling, input placeholder).

Proposed future work (not implemented)

Table 15. Proposed improvements. None of these exist in the current project.

Area	Proposal
Feature engineering	Ratios such as rooms per household and bedrooms per room; log transforms; location-based features
Model comparison	Additional boosting libraries and models; broader or randomized search extending beyond the grid's edges
Validation	Spatial or temporal hold-outs; repeated cross-validation; fixed random seeds for all models
Input validation	Range and category checks in the app; fix the placeholder and output-scale issues
Uncertainty	Prediction intervals (for example quantile or conformal methods)
Automated tests	Unit tests for preprocessing and the app; CI on every change
Model versioning	Track artifacts, parameters and metrics; record library versions
Deployment and monitoring	Public deployment with monitoring for data drift and performance

15 Conclusion

The project implements a complete regression workflow on the California Housing Prices data: EDA, a leakage-safe preprocessing pipeline, a Linear Regression baseline, a five-model cross-validated comparison, grid-search tuning, a held-out test evaluation, model serialization and a Streamlit interface. HistGradientBoostingRegressor was chosen on the lowest cross-validation RMSE (48,140.81). The final pipeline reaches a test RMSE of **47,319.30**, MAE of **31,253.32** and R^2 of **0.829**, compared with 70,059.19, 50,670.49 and 0.625 for the baseline.

These are solid results for a learning project, but they come from a single split of one historical dataset, errors grow for expensive areas, and the target is capped. The model should not be described as production-ready. Two concrete defects should be fixed before the application is used: the ×100,000 display scaling and the mismatch between the tuned and the final l2 setting.

16 References

[1] California Housing Prices dataset, Kaggle: https://www.kaggle.com/datasets/camnugent/california-housing-prices (link as cited in the project notebook and README; the dataset's original provenance is not stated in the project files).

[2] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[3] scikit-learn documentation: https://scikit-learn.org

[4] Streamlit documentation: https://docs.streamlit.io

[5] pandas: https://pandas.pydata.org; NumPy: https://numpy.org; Joblib: https://joblib.readthedocs.io

[6] Project files: README.md, house_price_prediction.ipynb, housing.csv, app.py, house_price_model.joblib, requirements.txt (author: Alwin V J).

A Appendix: Verification Log and Technical Inconsistencies

Findings from inspecting the project. Notebook outputs and executed behaviour take precedence over README statements. No original file was modified.

Table 16. *Inconsistencies and observations.*

#	Finding	Evidence	Impact
1	Final l2_regularization = 0.1, but GridSearchCV best is 0.0	Notebook output of best_params_; the “Retraining with best params” code; saved model parameters	Final model is not the exact GridSearchCV winner; practically tied in a supplementary re-run
2	App multiplies output by 100,000	app.py; notebook prediction 466,109.45; app displays $46,610,944,761.88	**High:** all displayed predictions overstated 100,000-fold
3	Final estimator (and the RF/HGB comparison runs) set no random_state; HGB early stopping is active	Notebook code; saved model random_state None, early stopping active	Retraining is not bit-for-bit reproducible
4	README says the final configuration uses l2 = 0.1 as “selected”	README “Hyperparameter Tuning” section versus notebook output	README presents the manual choice as the tuning result
5	Environment mismatch	requirements.txt pins pandas 2.3.3, scikit-learn 1.7.2; notebook output shows pandas 3.x types; artifact saved with scikit-learn 1.9.0; matplotlib and seaborn (used by the notebook) are not listed; file is UTF-16 encoded	Artifact loads under 1.7.2 with a version warning and predicted identically in the test
6	Placeholder category and out-of-range defaults accepted by the app	app.py sentinel check “None” never matches; all-zero one-hot confirmed; defaults give $30.8 billion	Invalid inputs yield a prediction silently
7	Documentation wording	Notebook says a higher longitude is farther west, but values are negative, so higher means further east; README lists “Streamlit Deployment” but no deployment exists	Minor; no effect on results
8	Dataset provenance	Only the Kaggle link is given	Original source and period could not be verified